

From ECDH to KEY_5 : landing a derived CMAC key in ICUSE

The one-sentence answer

The ECDH path *derives* the key — ephemeral P-224 keys → shared secret Z → X9.63/SHA-256 KDF → 16-byte AES-128 CMAC key — but as delivered it stops at software storage and RAM_KEY; to persist it in KEY_5 you must add one new step: wrap those 16 bytes in the SHE M1|M2|M3 update protocol (SecLib_KeySHEUpdate , keyId = 0x08) — the same machinery Lesson 0001 used for DATASTORE_KEY.

GENERATED VS. DERIVED — THE RETURN OF LESSON 0002'S DISTINCTION

DATASTORE_KEY was *generated* (TRNG). This key is *derived*: its value is a deterministic function of the ECDH shared secret plus the message-group ID. Both land in ICUSE the **same way** — M1|M2|M3 + CMD_LOAD_KEY — because once the 16 bytes exist, ICUSE doesn't care where they came from.

1 · The pipeline as implemented (docs/ecc_integration_notes.md)

INIT · NOTES §2

SecLib_EccInit() once after SecLib_Init() — RNG bridges to the ICUS PRNG; nothing else changes in the stack.



EPHEMERAL KEYPAIR · CRYPTO_30_CRYWRAPPER_HW.C:880

Hw_KeyExchangeCalcPubVal → SecLib_EccGenerateEphemeralKey : P-224 scalar d (28 B, unbiased in [1, n−1]), public point x||y (56 B). d parked in element CRYPTO_KE_KEYEXCHANGE_PRIVKEY — ICUS slots untouched.



EXCHANGE & SHARED SECRET · CRYPTO_30_CRYWRAPPER_HW.C:947

Peer point validated **on-curve** (invalid-curve hardening), then K = d · Q_peer via SecLib_EccComputeSharedSecret . Result into element SHAREDVALUE — 56 B (x||y) or 28 B (x-only) per config.



DERIVE · CRYPTO_30_CRYWRAPPER_HW.C:1037

Hw_KeyDerive : PASSWORD element = shared value, SALT element = message-group ID (1 B) → K_out = X9.63-KDF-SHA256(Z, SI) truncated to **16 bytes**. *This output is your new AES-128 CMAC key.*



AS DELIVERED · NOTES §6

Key lives in software KeyStorage (TmpKey → Csm_KeyCopy → group key element) and, in Phase 2, is activated into the volatile RAM_KEY via SecLib_KeyRAMUpdate . **Zero persistent slots — by design.**

The KDF is textbook ANSI X9.63 (seclib_ecc.c:293): per 32-byte block, S_i = SHA-256(Z || counter_{BE32} || SI), counter starting at 1, blocks concatenated and truncated. SHA-256 runs in software (mbedtls), same as the ECP math — ICUSE has no ECC engine, and the notes record that as an agreed assurance deviation (private keys in zeroized CPU RAM, never in NVM or a protected slot).

2 · The new step: KEY_5

Your requirement changes a design boundary the notes explicitly set ("persistent slots are never written by the ECC path"). Here is what landing the key in KEY_5 takes:

PARAMETER	VALUE	WHY / ANCHOR
Slot	KEY_5 → raw 0x08	Cry_30_Rh850Icus_Types.h:75; manual Table 4.1 (KEYMD = 0, no extended page)
keyId / authKeyId	KEY_5 / KEY_5	Self-authenticated first load — empty slot ⇒ KEY_AuthID ≡ 0 ¹²⁸ (manual §4.4). Alternative: authenticate with MASTER_ECU_KEY if one is provisioned.
Counter	1, then +1 per renewal	Monotonic anti-replay (manual §4.4). Matters more here — see the WP trap below.
Flags	KU — and think hard before WP	KU = CMAC-only, exactly the requirement. WP is a one-way latch...
Mechanism	SecLib_GenerateM1M2M3 + SecLib_KeySHEUpdate → CMD_LOAD_KEY	Reuse the Lesson-0001 machinery with keyId = KEY_5 ; nothing new is invented.
Prove it	CMAC generate + verify round-trip	SecSB_CreateKey 's tail (secure_boot.c:463–480), adapted.

THE WP TRAP — THE MOST IMPORTANT DESIGN DECISION HERE

If the ECDH key *rotates* (renegotiation per the MPKA lifecycle), a WP -flagged KEY_5 can **never be rewritten** — ICUSE rejects the update with ERC_KEY_WRITE_PROTECTED . DATASTORE_KEY earns WP because its value never changes. A rotating ECDH-derived key must either stay in volatile RAM_KEY (the delivered design), be loaded without WP (rewritable, protected by the protocol instead), or accept slot-in-place freeze. Decide before the first provisioning — WP cannot be undone.

Two concrete integration gaps in the current tree

- The SecLib switch doesn't know KEY_5.** SecLib_MACGenerate / SecLib_MACVerify whitelist slots: BOOT_MAC_KEY, SECUREBOOT_KEY, DATASTORE_KEY, INTEGRITY_KEY, KEY_RAM (crypto_library.c:238–246, 319–327). Using KEY_5 for CMAC hits default → "keyId not supported" . Add case KEY_5: to both.
- Bridge the layers.** The derived 16 bytes currently sit in a CryWrapper *key element* (software). To reach M1M2M3 you must read them out (e.g. SecLib_KeySHEUpdate called with a filled SecLib_KeyDataType , or a small new SecLib_StoreDerivedKeyInSHE(keyId, key16) in the security lib that composes Lesson-0001's flow) — mind the RAM_KEY ownership policy while doing so, and wipe the software copy after CMD_LOAD_KEY succeeds.

3 • Knowledge check

Q1 • On P-224, what are the private scalar and public point sizes?

- scalar 16 bytes, public 32 bytes
- scalar 32 bytes, public 64 bytes
- scalar 28 bytes, public 56 bytes
- scalar 24 bytes, public 48 bytes

seclib_ecc.h:37–39 — SECLIB_ECC_PRIVKEY_LEN 28, PUBKEY_LEN 56 (uncompressed X|Y, big-endian). P-224 coordinates are 224 bits = 28 bytes.

Q2 • Which KDF turns the shared secret into the CMAC key?

- HMAC-SHA256 counter mode, salt key
- ANSI X9.63 with SHA-256 hashing
- SHE KDF with KEY_UPDATE constants
- PBKDF2 with many SHA iterations

seclib_ecc.c:293 — Si = SHA-256(Z || counterBE32 || SI), SI = 1-byte message-group ID; output truncated to 16 B. The SHE KDF (Lesson

0001) is a different function, used only for M1M2M3 transport keys.

Q3 · How do the derived 16 bytes get into KEY_5?

CMD_LOAD_PLAIN_KEY targets slot five

KeyElementSet writes the ICU flash

M4M5 verification writes the slot

M1M2M3 protocol via CMD_LOAD_KEY

Persistent slots only accept the SHE update protocol: SecLib_GenerateM1M2M3 + SecLib_KeySHEUpdate → CMD_LOAD_KEY (Lesson 0001). PLAIN_KEY is RAM_KEY-only; KeyElementSet is software storage; M4M5 is the read-back proof, not a write path.

Q4 · Why is setting WP on KEY_5 dangerous for a rotating key?

renegotiated keys can never be written

WP forces the key into RAM instead

WP disables CMAC verification entirely

WP erases the slot on every reset

WP is a one-way latch (manual §4.3): the slot can never be rewritten or erased. The next ECDH negotiation produces a new key that KEY_5 would refuse — ERC_KEY_WRITE_PROTECTED.

4 · Read next / practice

PRIMARY SOURCE

docs/ecc_integration_notes.md §3 and §6 — the authoritative map of what was delivered, the key-element semantics, and the storage boundaries. Then ICUSE manual §4.4 for the M1M2M3 form the new step must produce.

- Fact sheet: ECDH → ICUSE slot pipeline — the whole chain, the KEY_5 parameter set, and the integration checklist.
- Practice: sketch the SecLib_KeyDataType struct (crypto_library_type.h:116–125) filled for KEY_5: keyId, authKeyId, UID, counter, flags, key, authKey. Check yourself against secure_boot.c:437–447.
- Review: Lesson 0001 (M1M2M3), Lesson 0002 (generated vs derived).

YOUR TEACHER

The WP decision, the SecLib switch extension, the RAM_KEY ownership policy, the assurance deviation — if any of these feel shaky, ask. This is exactly the point where design mistakes get expensive.